



PRÉSENTATION TECHNIQUE

Manipuler les variables en PHP

Créer, modifier, combiner et comprendre les variables

Guide simple et détaillé — Sonatel Academy
Préparé pour Khadija — Dakar, Sénégal

1. Qu'est-ce qu'une variable ?

DÉFINITION

Une variable est un espace mémoire auquel on donne un nom, et qui sert à **stocker une information** pour pouvoir la réutiliser, la modifier ou l'afficher plus tard dans le programme.

ANALOGIE SIMPLE

Une variable est comme une boîte de rangement avec une étiquette collée dessus. L'étiquette, c'est le **nom** de la variable. Le contenu de la boîte, c'est la **valeur** stockée. Tu peux ouvrir la boîte, regarder ce qu'il y a dedans, et même remplacer le contenu par autre chose, sans changer l'étiquette.

Comment déclarer une variable en PHP

En PHP, toute variable commence obligatoirement par le symbole `$`, suivi du nom que tu choisis.

```
<?php
    $prenom = "Khadija";
?>
```

Ici, on lit cette ligne comme : "Je crée une variable nommée `prenom`, et je lui donne la valeur `Khadija`." Le symbole `=` ne veut pas dire "égal" comme en mathématiques, il veut dire **"on affecte"** ou **"on assigne"**.

Les règles à respecter pour nommer une variable

- Le nom doit toujours commencer par `$`, puis une lettre ou un underscore `_` (jamais directement par un chiffre).
- Le nom peut contenir des lettres, des chiffres et des underscores, mais pas d'espaces ni de caractères spéciaux comme `-` ou `é`.
- PHP fait la différence entre majuscules et minuscules : `$nom` et `$Nom` sont deux variables différentes.
- Il est recommandé de choisir un nom clair qui décrit ce que contient la variable (`$age` plutôt que `$a`).

Nom de variable	Valide ?	Pourquoi
<code>\$age</code>	✔ Oui	Commence par une lettre
<code>\$_total</code>	✔ Oui	Commence par un underscore
<code>\$nom2</code>	✔ Oui	Chiffre autorisé après une lettre
<code>\$2nom</code>	✘ Non	Ne peut pas commencer par un chiffre
<code>\$mon-nom</code>	✘ Non	Le tiret n'est pas autorisé

2. Modifier et afficher une variable

Modifier la valeur d'une variable

Une variable n'est jamais figée : on peut changer sa valeur autant de fois qu'on veut pendant l'exécution du programme.

```
<?php
    $score = 10;
    echo $score;    // Affiche : 10

    $score = 25;    // on remplace la valeur
    echo $score;    // Affiche : 25
?>
```

ASTUCE PÉDAGOGIQUE

Pour bien faire comprendre cela à tes camarades : dis-leur que PHP lit le code **du haut vers le bas, ligne par ligne**. À chaque fois qu'il rencontre `$score = ...`, il efface l'ancienne valeur et met la nouvelle à la place. Ce n'est jamais les deux valeurs en même temps.

Afficher une variable avec echo

L'instruction `echo` permet d'afficher le contenu d'une variable à l'écran. On peut afficher une variable seule, ou la combiner avec du texte.

```
<?php
    $ville = "Dakar";

    echo $ville;                // Affiche : Dakar
    echo "J'habite à " . $ville; // Affiche : J'habite à Dakar
?>
```

DÉFINITION

Le point `.` en PHP s'appelle **l'opérateur de concaténation**. Il sert à "coller" du texte et des variables ensemble pour former une seule chaîne de caractères.

Une autre façon d'afficher : l'interpolation

Quand le texte est entre **guillemets doubles** `" "`, PHP est capable de reconnaître automatiquement une variable insérée directement dans la phrase, sans avoir besoin du point de concaténation.

```
<?php
    $prenom = "Khadija";

    echo "Bonjour $prenom !"; // Affiche : Bonjour Khadija !
    echo 'Bonjour $prenom !'; // Affiche : Bonjour $prenom ! (ne marche pas)
?>
```

PIÈGE FRÉQUENT

Avec des **guillemets simples** `' '`, PHP n'interprète pas la variable : il affiche le texte tel quel, y compris le symbole `$`. Cette différence entre guillemets simples et doubles est une question classique de débutant.

3. Faire des opérations sur les variables

Les opérations mathématiques de base

PHP permet de manipuler des variables numériques comme dans une calculatrice classique.

```
<?php
    $a = 10;
    $b = 3;

    echo $a + $b;    // Addition      → 13
    echo $a - $b;    // Soustraction → 7
    echo $a * $b;    // Multiplication→ 30
    echo $a / $b;    // Division      → 3.33
    echo $a % $b;    // Modulo (reste)→ 1
?>
```

DÉFINITION

Le **modulo** (symbole `%`) donne le reste d'une division entière. Par exemple 10 divisé par 3 = 3 reste 1, donc `10 % 3` vaut 1. C'est très utilisé pour savoir si un nombre est pair ou impair.

Les opérateurs combinés (raccourcis)

PHP propose des raccourcis pratiques pour modifier une variable à partir de sa propre valeur, sans avoir à la réécrire deux fois.

Écriture longue	Raccourci	Effet
<code>\$x = \$x + 1;</code>	<code>\$x++;</code>	Ajoute 1 à \$x
<code>\$x = \$x - 1;</code>	<code>\$x--;</code>	Retire 1 à \$x
<code>\$x = \$x + 5;</code>	<code>\$x += 5;</code>	Ajoute 5 à \$x
<code>\$x = \$x - 5;</code>	<code>\$x -= 5;</code>	Retire 5 à \$x
<code>\$x = \$x . "!";</code>	<code>\$x .= "!";</code>	Ajoute "!" au texte de \$x

```
<?php
    $compteur = 0;
    $compteur++;      // $compteur vaut maintenant 1
    $compteur += 10;   // $compteur vaut maintenant 11

    $message = "Bonjour";
    $message .= " Khadija"; // "Bonjour Khadija"
?>
```

4. Le type d'une variable et sa portée

PHP devine automatiquement le type

Contrairement à d'autres langages, en PHP tu n'as pas besoin de préciser le type d'une variable (texte, nombre...). PHP le devine tout seul selon la valeur que tu lui donnes. On appelle cela un **typage dynamique**.

```
<?php
$a = "5";           // PHP comprend : c'est du texte (string)
$b = 5;             // PHP comprend : c'est un nombre entier (integer)
$c = 5.5;           // PHP comprend : c'est un nombre décimal (float)

echo gettype($a);   // Affiche : string
echo gettype($b);   // Affiche : integer
?>
```

ASTUCE PÉDAGOGIQUE

La fonction `gettype()` est très utile pour vérifier le type réel d'une variable quand on débute, ou pour expliquer ce concept à tes camarades en testant plusieurs exemples en direct.

Vérifier si une variable existe

Avant d'utiliser une variable, on peut vérifier qu'elle a bien été créée avec la fonction `isset()`, qui renvoie `true` ou `false`.

```
<?php
$nom = "Khadija";

if (isset($nom)) {
    echo "La variable existe.";
}

if (isset($age)) {
    echo "Cette ligne ne s'affichera pas.";
}
?>
```

La portée des variables (scope)

DÉFINITION

La **portée** d'une variable désigne l'endroit du code où cette variable est "visible" et utilisable. Une variable créée à l'intérieur d'une fonction n'existe que dans cette fonction, elle n'est pas accessible en dehors.

```
<?php
function direBonjour() {
    $message = "Bonjour !"; // existe seulement ici
    echo $message;
}

direBonjour();           // Affiche : Bonjour !
echo $message;           // Erreur : $message n'existe pas ici
?>
```

PIÈGE FRÉQUENT

Beaucoup de débutants pensent qu'une variable créée n'importe où est utilisable partout. C'est faux : une variable créée à l'intérieur d'une fonction reste "enfermée" dans cette fonction, sauf si on la retourne explicitement avec `return`.

5. Résumé pour bien expliquer les variables

Action	Exemple de code
Créer une variable	<code>\$nom = "Khadija";</code>
Modifier une variable	<code>\$nom = "Moussa";</code>
Afficher une variable	<code>echo \$nom;</code>
Combiner texte et variable	<code>echo "Salut " . \$nom;</code>
Variable dans une chaîne	<code>echo "Salut \$nom";</code>
Augmenter de 1	<code>\$x++;</code>
Vérifier le type	<code>gettype(\$nom);</code>
Vérifier l'existence	<code>isset(\$nom);</code>

La phrase à retenir pour ta présentation

"Une variable en PHP, c'est une boîte étiquetée avec un symbole \$ devant son nom. On peut y ranger une information, la lire avec echo, la modifier à tout moment, et PHP devine automatiquement de quel type elle est, sans qu'on ait besoin de le préciser."

Erreurs courantes à éviter

- Oublier le symbole `$` devant le nom de la variable.
- Confondre guillemets simples `' '` (texte brut) et guillemets doubles `" "` (variables interprétées).
- Oublier le point `.` pour assembler texte et variable.
- Croire qu'une variable créée dans une fonction est utilisable partout dans le fichier.
- Oublier le point-virgule `;` à la fin de chaque instruction.